SLAVE SERIAL CONFIGURATION MANAGER FOR VIRTEX-5 FPGA

Report submitted for completion of training

Programme of Space Applications Centre

Submitted by
AYUSH SINGHAL
(Registration No. RS03440)
B.Tech. (Electronics and Communication Engineering)

Under the guidance of

Sri. Himanshu N Patel
HEAD, SCI/ENGR-G
MRSA-MSDG-MSCED

Sri. Avadhesh Kumar
SCI/ENGR-SE
MRSA-MSDG-MSCED

Space Applications Centre, (ISRO) Ahmedabad

Institution
Pandit Deendayal Energy University
Gandhinagar – 382426
Gujarat

SRTD-RTMG-MISA
Space Applications Centre (ISRO)
Ahmedabad, Guajrat

13 May 2026 to 20 July 2026

SLAVE SERIAL CONFIGURATION MANAGER FOR VIRTEX-5 FPGA

A PROJECT REPORT

Submitted by

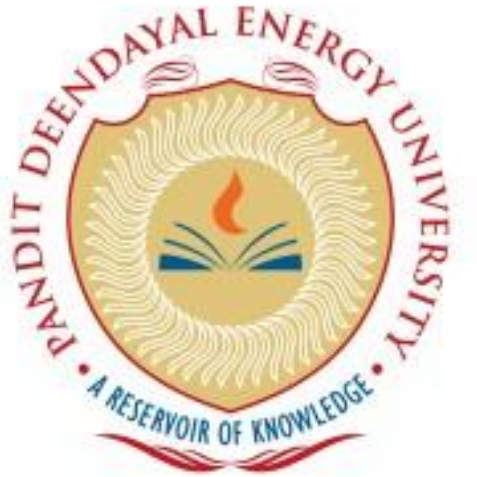
Ayush Singhal

In partial fulfillment for the award of the degree of

BACHELOR OF ENGINEERING

In

Electronics and Communication Engineering



Pandit Deendayal Energy University, Gandhinagar

13 May 2026 to 20 July 2026

ACKNOWLEDGEMENT

I would like to acknowledge the contribution of certain distinguished people, without their support and guidance, this project would not have been completed.

Firstly, I would like to thank SAC-ISRO, Ahmedabad for letting me to do the project work in such an esteemed organization as theirs.

I take this opportunity to thank **Dr. M.B. Mahajan, DEPUTY DIRECTOR (MRSA), OUTSTANDING SCI.** , **Shri B. Saravana Kumar, GROUP DIRECTOR (MRSA/MSDG) SCI/ENGR-G, Shri Himanshu N Patel, HEAD (MRSA/MSDG/MSCED) SCI/ENGR-G** and **Shri Avadhesh Kumar, SCI/ENGR-SE (MRSA/MSDG/MSCED), SAC-ISRO** for guiding me during the project and help me gain first-hand experience of new technologies.

I would also like to thank all the scientists, engineers, technical staff, and colleagues at SAC-ISRO who directly or indirectly contributed to this project by providing valuable discussions, technical insights, and a supportive working environment throughout my internship.

I would like to express my sincere thanks and deep sense of gratitude to my mentor, **Prof. Dr. Manish Kumar**, for his guidance and moral support during the course of preparation of this project report. I really thank her sincerely for always being there with her extreme knowledge and kind nature. In addition, very thankful to our **H.O.D Dr. Parth Thakkar.**

ABSTRACT

In this report, we will present the design and hardware realization of a proprietary and robust Slave Serial Bootloader and Configuration Engine for Xilinx Virtex-5 FPGA. The design was completed during an internship at Space Application Centre (SAC), ISRO. The system is a custom connection between a proprietary Honeywell 64 Megabits (8 MB) MRAM piggyback module and the configuration port of the Virtex-5.

I coded this architecture using Verilog RTL and consists of three tightly interconnected modules namely; an intake pipeline of the MRAM with integrated 2-to-4 Die decoder, a double buffered Parallel-In-Serial-Out (PISO) serializer for efficient byte transfer, and a master Executive Boot Finite State Machine (FSM).

For the hardware implementation, the system operates at a main design clock frequency of 3 MHz, while the actual configuration process runs at 1.5 MHz to stream the data. The waveforms observed on the oscilloscope confirmed that the clock network functions correctly.

The final hardware runs that data bytes transfer from the MRAM to the target FPGA, confirming the entire boot loop works as intended.

TABLE OF CONTENTS

ACKNOWLEDGEMENT **3**

ABSTRACT **4**

CHAPTER 1: INTRODUCTION **6**

1.1 PROBLEM STATEMENT **6**

1.2 INTRODUCTION TO FPGA: **6**

1.2.1 FPGA TECHNOLOGY: 6

1.2.2 ARCHITECTURE OF FPGA: 8

1.2.3 FPGA DESIGN AND PROGRAMMING: 8

1.2.4 APPLICATIONS OF FPGAS: 9

1.3 OVERVIEW OF PROJECT: **11**

CHAPTER 2 : COMPONENTS **12**

2.1 SOFTWARE COMPONENTS: **12**

2.2 HARDWARE COMPONENTS: **12**

CHAPTER 3: PROJECT IMPLEMENTATION **13**

3.1 SOFTWARE SIMULATION: **13**

3.2 HARDWARE IMPLEMENTATION: **15**

CHAPTER 4: SIMULATION RESULTS **16**

CHAPTER 5: CHALLENGES FACED **17**

CHAPTER 6: CONCLUSION AND FUTURE SCOPES **18**

REFERENCES **19**

ANNEXURE **20**

CHAPTER 1: INTRODUCTION

1.1 PROBLEM STATEMENT

For this project I used a custom Honeywell 64 Mb module. This Honeywell 64 Mb module has four stacked 16 Mb dies. I put this Honeywell 64 Mb module on a motherboard. The problem was that the motherboard did not have hardware decoding. This meant that the signal routing was not normal.

I had to design a metal RTL bootloader for the Xilinx Virtex-5 FPGA. This RTL bootloader for the Xilinx Virtex-5 FPGA had to do a lot of things. It had to select the die from the Honeywell 64 Mb module. It had to make sure that the MRAM was accessed safely so the Honeywell 64 Mb module would work properly. It had to prevent the MRAM from being written to. It had to configure the Xilinx Virtex-5 FPGA every time.

1.2 INTRODUCTION TO FPGA:

1.2.1 FPGA TECHNOLOGY:

A field-programmable gate array (FPGA) is an integrated circuit designed to be configured by a customer or a designer after manufacturing—hence "field-programmable". The FPGA configuration is generally specified using a hardware description language (HDL), similar to that used for an application-specific integrated circuit (ASIC).

Contemporary FPGAs have large resources of logic gates and RAM blocks to implement complex digital computations. As FPGA designs employ very fast I/Os and bidirectional data buses it becomes a challenge to verify correct timing of valid data within setup time and hold time. Floor planning enables resources allocation within FPGA to meet these time constraints. FPGAs can be used to implement any logical function that an ASIC could perform. The ability to update the functionality after shipping, partial re-configuration of a portion of the design and the low non-recurring engineering costs relative to an ASIC design offer advantages for many applications.

FPGAs contain programmable logic components called "logic blocks", and a hierarchy of reconfigurable interconnects that allow the blocks to be "wired together"—somewhat like many (changeable) logic gates that can be inter-wired in (many) different configurations. Logic blocks can be configured to perform complex combinational functions, or merely simple logic

gates like AND and XOR. In most FPGAs, the logic blocks also include memory elements, which may be simple flip-flops or more complete blocks of memory.

Some FPGAs have analog features in addition to digital functions. The most common analog feature is programmable slew rate and drive strength on each output pin, allowing the engineer to set slow rates on lightly loaded pins that would otherwise ring unacceptably, and to set stronger, faster rates on heavily loaded pins on high-speed channels that would otherwise run too slowly. Another relatively common analog feature is differential comparators on input pins designed to be connected to differential signaling channels. A few "mixed signal FPGA's have integrated peripheral analog-to-digital converters (ADCs) and digital-to-analog converters (DACs) with analog signal conditioning blocks allowing them to operate as a system-on-a-chip. Such devices blur the line between an FPGA, which carries digital ones and zeros on its internal programmable interconnect fabric, and field-programmable analog array(FPAA), which carries analog values on its internal programmable interconnect fabric.

Applications of FPGAs include digital signal processing, software-defined radio, cryptography,bioinformatics, computer hardware emulation, speech-recognition, radio astronomy, metal detection and a growing range of other areas. FPGAs originally began as competitors to CPLDs and competed in a similar space, that of glue logic for PCBs. As their size, capabilities, and speed increased, they began to take over larger and larger functions to the state where some are now marketed as full systems on chips (SoC). Particularly with the introduction of dedicated multipliers into FPGA architectures in the late 1990s, applications which had traditionally been the sole reserve of DSPs began to incorporate FPGAs instead.

Traditionally, FPGAs have been reserved for specific vertical applications where the volume of production is small. For these low-volume applications, the premium that companies pay in hardware costs per unit for a programmable chip is more affordable than the development resources spent on creating an ASIC for a low-volume application. Today, new cost and performance dynamics have broadened the range of viable applications.

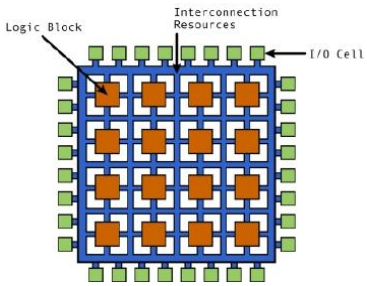


Figure 0.1 Generic FPGA Architecture

1.2.2 ARCHITECTURE OF FPGA:

The architecture consists of configurable logic blocks, configurable I/O blocks, and programmable interconnect. Also, there will be clock circuitry for driving the clock signals to each logic block. Additional logic resources such as ALUs, memory, and decoders may also be available. The three basic types of programmable elements for an FPGA are static RAM, anti-fuses, and flash EPROM.

1.2.3 FPGA DESIGN AND PROGRAMMING:

To define the behaviour of the FPGA, the user provides a hardware description language (HDL) or a schematic design. The HDL form is more suited to work with large structures because it's possible to just specify them numerically rather than having to draw every piece by hand. However, schematic entry can allow for easier visualisation of a design.

Then, using an electronic design automation tool, a technology-mapped netlist is generated. The netlist can then be fitted to the actual FPGA architecture using a process called place-and-route, usually performed by the FPGA Company's proprietary place-and-route software. The user will validate the map, place and route results via timing analysis, simulation, and other verification methodologies. Once the design and validation process is complete, the binary file generated (also using the FPGA company's proprietary software) is used to (re)configure the FPGA. This file is transferred to the FPGA/CPLD via a serial interface (JTAG) or to an external memory device like an EEPROM.

The most common HDLs are VHDL and Verilog, although in an attempt to reduce the complexity of designing in HDLs, which have been compared to the equivalent of assembly languages, there are moves to raise the abstraction level through the introduction of alternative languages. National Instruments 'LabVIEW graphical programming language (sometimes referred to as "G") has an FPGA add-in module available to target and program FPGA hardware. To simplify the design of complex systems in FPGAs, there exist libraries of predefined complex functions and circuits that have been tested and optimized to speed up the design process. These predefined circuits are commonly called IPcores, and are available from FPGA vendors and third-party IP suppliers (rarely free, and typically released under proprietary licenses). Other predefined circuits are available from developer communities such as Open Cores (typically released under free and open source licenses such as the GPL, BSD or similar license), and other sources.

In a typical design flow, an FPGA application developer will simulate the design at multiple stages throughout the design process. Initially the RTL description in VHDL or Verilog is

simulated by creating test benches to simulate the system and observe results. Then, after the synthesis engine has mapped the design to a netlist, the netlist is translated to a gate level description where simulation is repeated to confirm the synthesis proceeded without errors. Finally the design is laid out in the FPGA at which point propagation delays can be added and the simulation run again with these values back-annotated onto the netlist.

1.2.4 APPLICATIONS OF FPGAS:

Aerospace and Defence

- Communications
- Missiles & Munitions
- Secure Solutions
- Space

Audio

- Connectivity Solutions
- Portable Electronics
- Radio
- Digital Signal Processing (DSP)

Automotive

- High Resolution Video
- Image Processing
- Vehicle Networking and Connectivity
- Automotive Infotainment

Broadcast

- Real-Time Video Engine
- Edge-QAM
- Encoders
- Displays
- Switches and Routers

Consumer Electronics

- Digital Displays
- Digital Cameras

- Multi-function Printers
- Portable Electronics
- Set-top Boxes

High Performance Computing

- Servers
- Super Computers
- SIGINT Systems
- High-end RADARS
- High-end Beam Forming Systems
- Data Mining Systems

Industrial

- Industrial Imaging
- Industrial Networking
- Motor Control
- Industrial Imaging
- Secure Solutions
- Image Processing

Video & Image Processing

- High Resolution Video
- Video Over IP Gateway
- Digital Displays
- Industrial Imaging

Communications

- Optical Transport Networks
- Network Processing

1.3 OVERVIEW OF PROJECT:

- I designed a Virtex-5 Bootloader which is a data pipeline that organizes all data into three categories and consists of three types of very synchronous hardware modules. These modules and the three categories are integrated using a top-level wrapper.
- The first module is the MRAM Intake Engine. This module sweeps the 24 bits that make up a memory address. It is combined with a hard wired 2 to 4 decoder that decodes the top bits A[22:21] and chooses which MRAM die to activate. This is done using the active-low Chip Enable method. The die are selected at 2MB boundaries and are arranged in a stack of 16MB. This module also employs a handshake flag (tx_buffer_full) and a hardware locking mechanism (active_mission) to halt the pointer and prevent data overwrites.
- The next module is the PISO Serializer. This module is designed to maintain the clock and transfer address data with the MRAM interface. It also contains a configuration clock (v5_cclk) that controls the serial data to ensure that the Virtex-5 device (FPGA) samples data from v5_din at positive edge of v5_cclk and change data bits of bytes at negative edge of v5_cclk. To keep this serial data moving without any pauses, the module uses a shadow buffer. This buffer holds the next data byte from memory while the main register is busy shifting out the current bits, instantly loading the new byte as soon as the shifting finishes.
- The final module is the Executive Boot FSM & Top Wrapper (v5_bootloader_top) module which controls the entire booting process. This module also starts the Virtex-5 booting process by pulling the PROG_B signal low (active). This method completely erases the previous configuration. After the erasure, this module also puts the remaining system in a wait. The system will only exit the wait state after start_boot is high.

CHAPTER 2 : COMPONENTS

2.1 SOFTWARE COMPONENTS:

- Xilinx ISE Design Suite 14.7: Primary legacy EDA toolchain used for Virtex-5 place-and-route flows. Supports synthesis, translation, mapping, etc. Vivado does not support these newer flows; therefore, ISE 14.7 is required for these legacy EDA tools.

- ISim Simulator: Executes cycle-accurate behavioral simulations. It is an HDL simulation engine. Functional sweeps of the Verilog modules/HDL constructs prior to netlisting are conducted using this simulation engine.

- iMPACT Software: Configuring and programming devices with the ISE toolchain. Used for configuring the JTAG boundary-scan chain, applying the constraints, and downloading/fleshing the bitstream on the hardware.

2.2 HARDWARE COMPONENTS:

Target Hardware Environment

- Heavy-Duty Prototyping Systems Board: A robust conduction cooled evaluation PCB encased in an aluminum shell integrated with structural card rails.

- Xilinx Virtex-5 (XC5V / Rad-Hard Virtex-5QV): The target device configured to physically process the incoming serial bit stream, controls the v5_prog_b and v5_init_b hardware handshaking lines and executes hardware startup sequences on the v5_done signal assertion.

- Honeywell HXNV06400 64-Megabit MRAM: A radiation hardened parallel non-volatile memory device providing the 8MB of memory.

- Ceramic/Hermetic IC Packages (CCGA/CQFP): Radiation hardened silicon packages with metal lids and with gold plating, for the Virtex-5 FPGA and the host Controller.

- High-Density Gold Plated Connectors: Robust multi-pin connectors designed to withstand high vibration and thermal stress.

- Discrete Manual Rework Matrix: A dense cluster of blue wire wrap jumper wires point to point routed to provide external access to internal FPGA logic.

Instrumentation & Debugging Tools

- Agilent Technologies Infiniium Series Scope: An advanced mixed signal storage oscilloscope, used to observe the real-time signal alignment of v5_cclk, v5_din, and dbg_byte_pulse.

- Xilinx Platform Cable USB II: The red hardware JTAG pod to load the synthesizable bootloader configuration bitstream, through the Xilinx ISE iMPACT.

- High-Bandwidth Coaxial Probes: Low capacitance oscilloscope probes, significally less than those of standard probes, to minimize signal distortion.

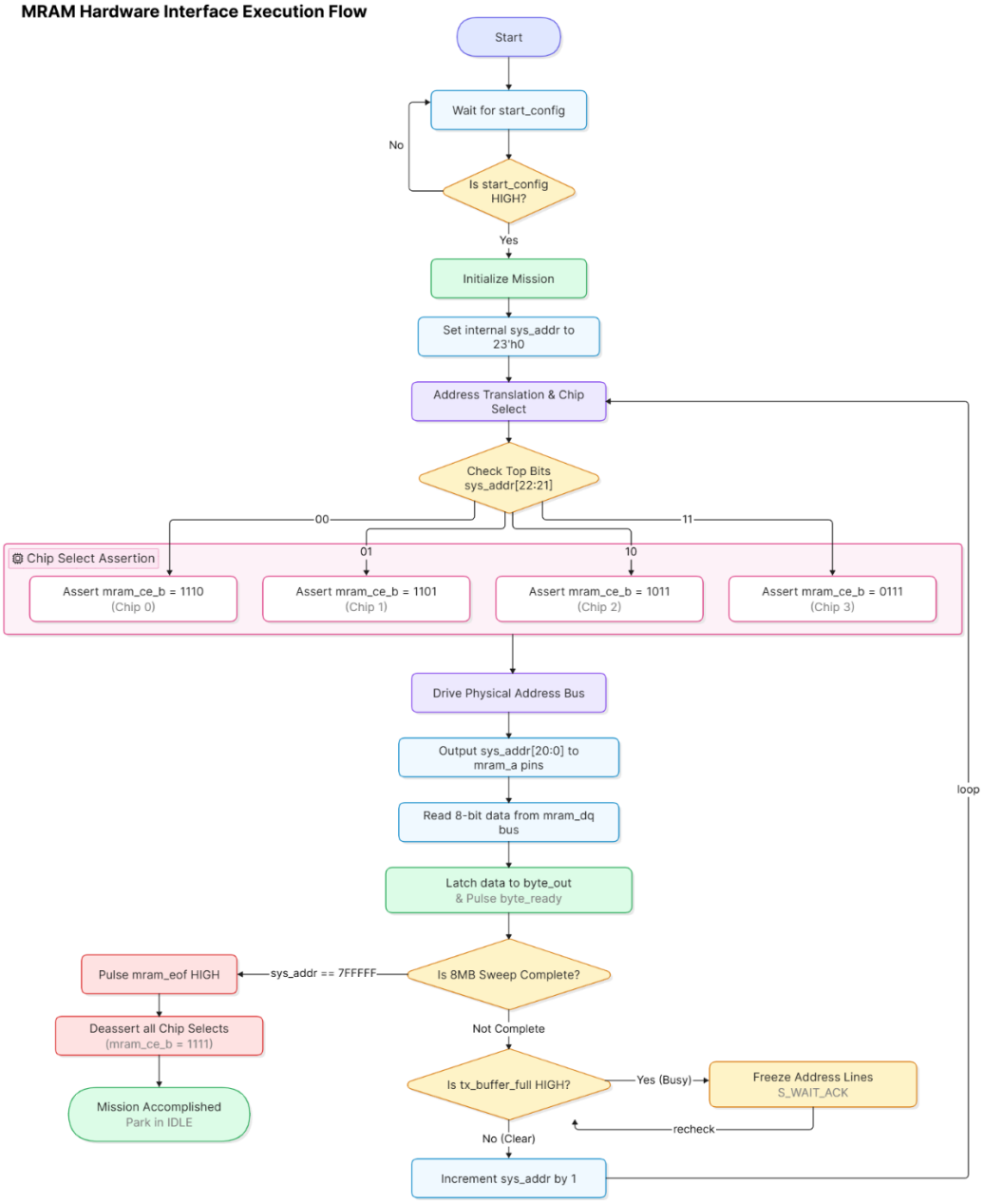
- Anti-Static (ESD) Grounding Strap: A blue electrostatic discharge wrist strap that ensures component safety when handling high-cost aerospace grade ceramic silicon.

CHAPTER 3: PROJECT IMPLEMENTATION

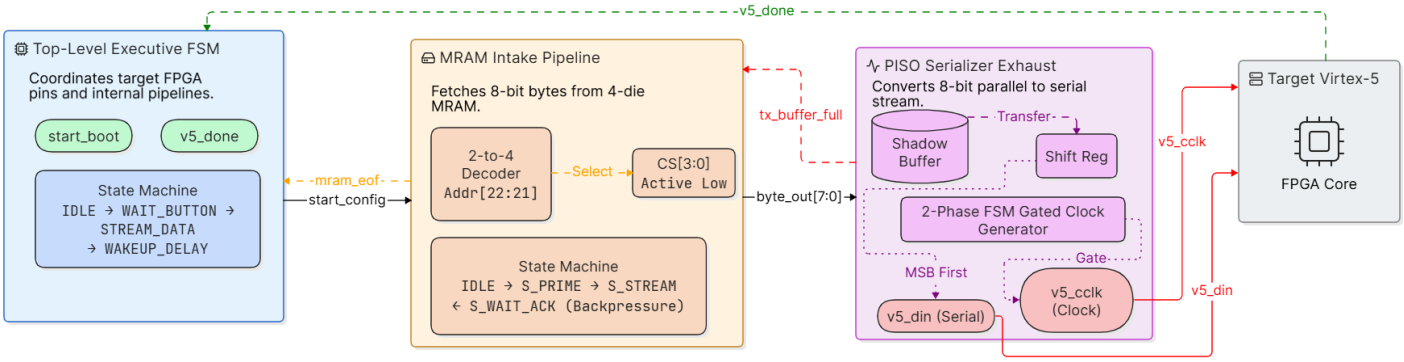
3.1 SOFTWARE SIMULATION:

PROJECT BLOCK DIAGRAMS:

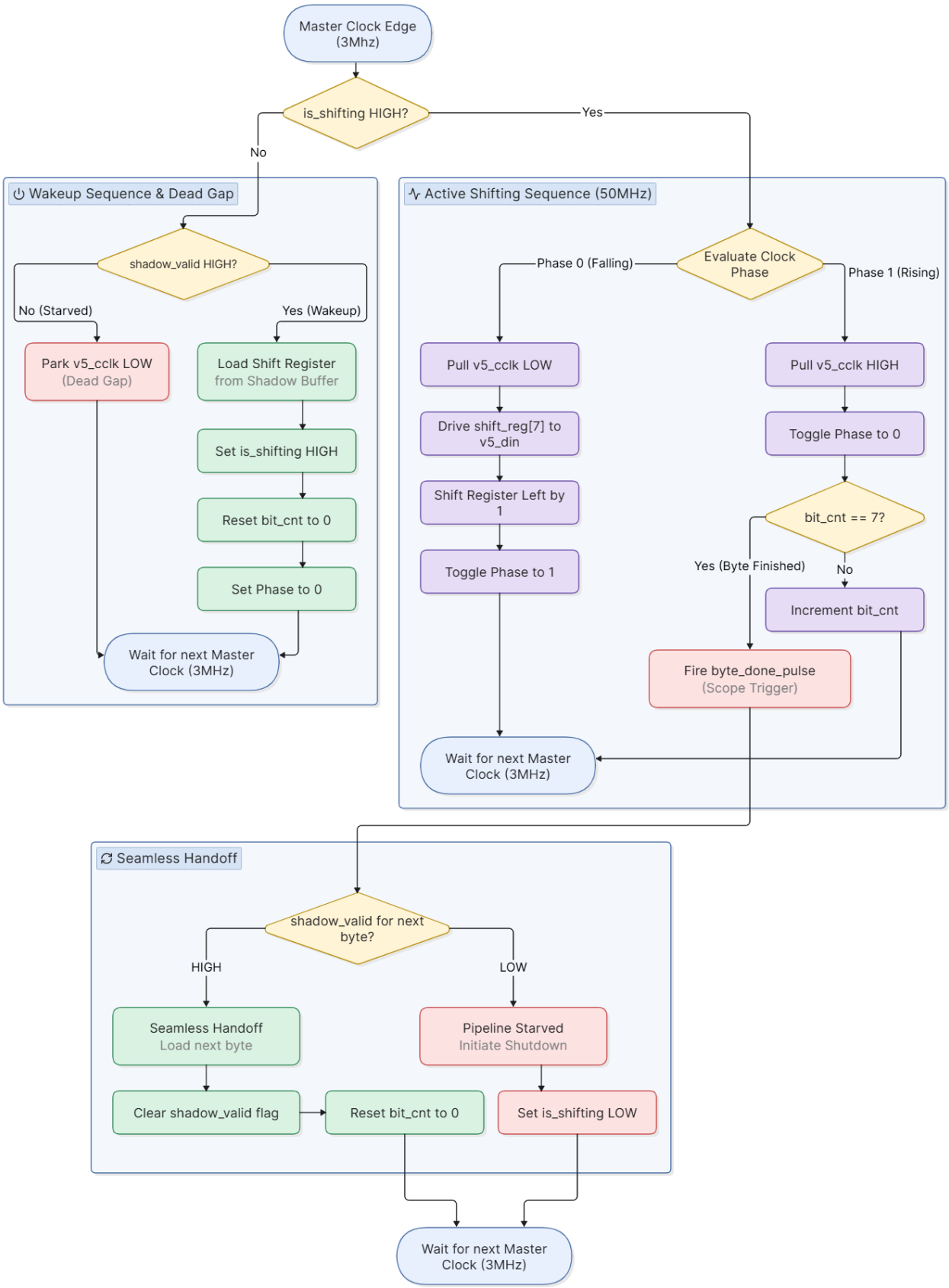
MRAM Hardware Interface Execution Flow



Slave Serial Configuration Manager for Virtex-5 FPGA (Whole System)



PISO Serializer Double-Buffered Hardware Engine



3.2 HARDWARE IMPLEMENTATION:

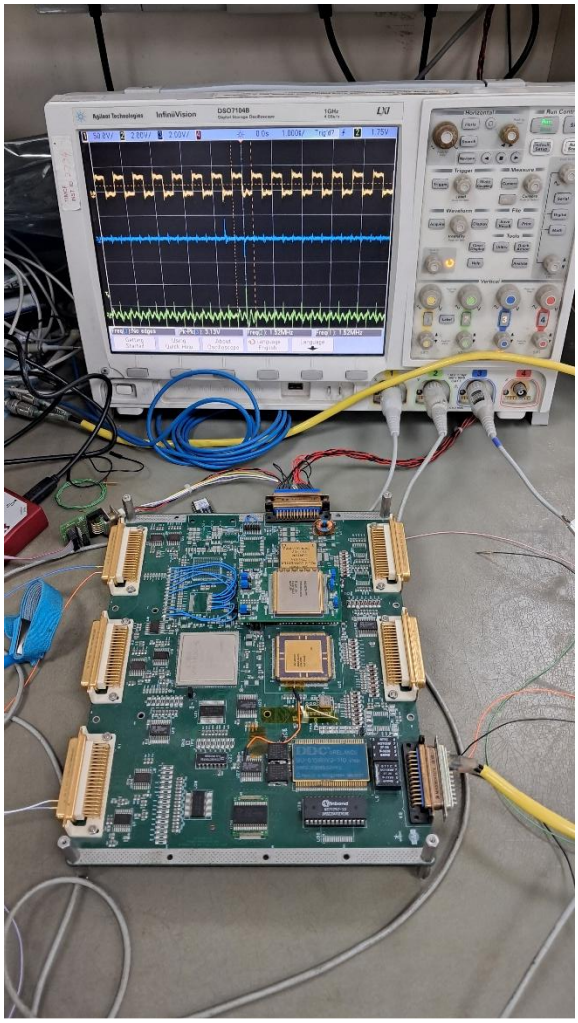


Fig 1

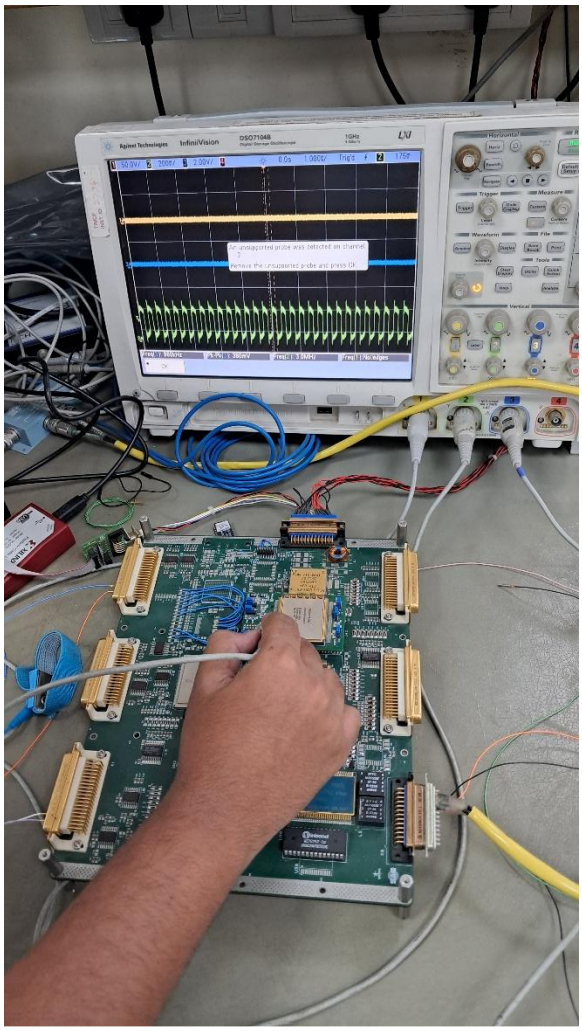
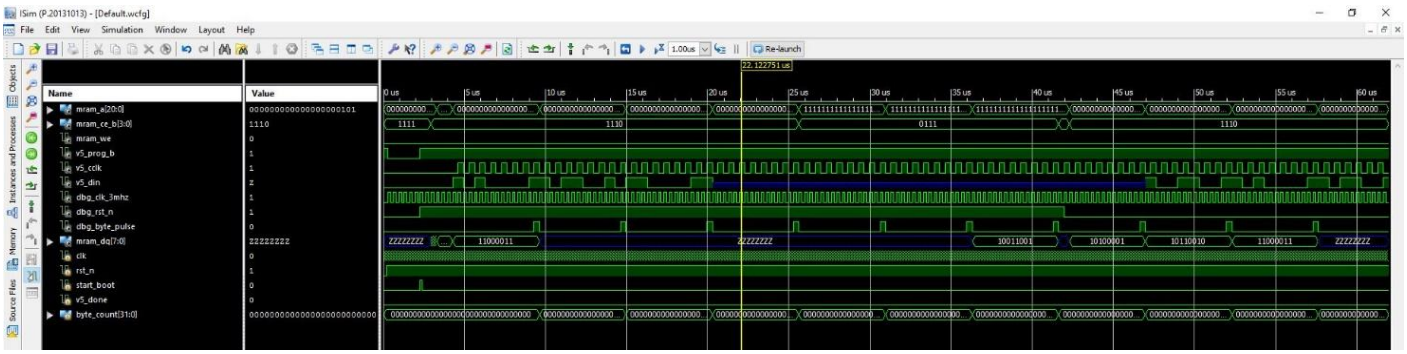


Fig 2

The physical verification process utilizes two separate oscilloscope profiles in order to separate hardware problems at the board level from problems in the internal RTL logic. In the first verification profile (Fig 1), the oscilloscope captures the gated configuration clock (v5_cclk) toggling at a constant 1.5 MHz. Even though the serial data line (v5_din) does not change, the clock gating logic confirms that the PISO is functioning properly. In the second verification profile (Fig 2), the divided clock (clk_3MHz) is verified as operating at a 3.0 MHz with a steady signal. Although the verification of frequency alone does not confirm the downstream FSM is operating nor does it confirm MRAM is being read, it does make us sure that clock divider network has been verified and that it is providing the synchronous timing to the overall system.

CHAPTER 4: SIMULATION RESULTS

Waveform of Top wrapper bootloader:



Boot Initiation: This process starts when the start_boot signal pulses high. This causes the v5_prog_b signal to go from low to high, and dbg_rst_n which is mram_enable will go low to high and starts MRAM interface to fetch data bytes from MRAM.

Memory Chip Selection: The internal 2-to-4 decoder changes the chip enable (mram_ce_b) lines to select the correct MRAM die. For example, the signal shifts from 1110 (activating the first die) to 0111 (activating a fourth die) while the address lines (mram_a) update to track the memory location.

Parallel Data Reading: The selected MRAM chip places 8-bit parallel data bytes onto the mram_dq[7:0] bus. We can see specific data bytes like (A1) 10100001 , (B2) 10110010, (C3) 11000011, (99) 10011001 appearing on the bus as the reads occur.

Data Serialization: The Parallel-In-Serial-Out (PISO) serializer takes these 8-bit parallel bytes from the MRAM and converts them into a 1-bit serial stream on the v5_din pin.

Byte Completion Indicator: The dbg_byte_pulse signal spikes high for a brief moment every time the serialization of a full 8-bit byte finishes. This tells the system that the current byte is completely sent out and the next one is ready.

Memory End Shutdown: The dbg_rst_n signal, which works as the mram_enable, drops from high to low as soon as the system reaches the very end of the MRAM data. This action turns off the memory interface to stop the process once the configuration is done.

Clock Frequency Relation: The simulation confirms the exact clock speeds requested. The main system clock (dbg_clk_3mhz) runs at 3 MHz, while the configuration clock (v5_cclk) runs at exactly half that speed (1.5 MHz) to safely push the serial data bits into the FPGA one by one.

CHAPTER 5: CHALLENGES FACED

1. Physical Pin Hijacking:

The schematic trace for the PMC connector shows that the MRAM die's four Chip Select lines are routed to the random address pins rather than their specific digital pins (e.g., A21, A22).

Resolution: Physical constraints are changed in the UCF to reduce the physical address bus to 21 bits and a custom combinational 2-to-4 decoder is designed and implemented within the FPGA to generate the physical Chip Select signals.

2. RTL Latching Faults in High-Speed Discrete Event Tracking:

To monitor byte-level processing on a physical oscilloscope, a diagnostic signal (byte_done_pulse) was integrated into the system. The initial implementation failed to define a default low-state assignment for the register at the evaluation block's boundary. In hardware logic, this oversight meant the register retained its asserted state indefinitely after the first transaction, transforming a intended 1-clock-cycle pulse into a permanent DC flatline.

Resolution: A synchronous default clearance assignment (byte_done_pulse <= 1'b0;) was introduced at the absolute boundary of the execution block. This guarantees that the diagnostic pin dynamically drops back to ground on the next clock edge, generating a true, distinct edge pulse for every single byte processed.

3. High-Speed Hardware Visibility:

The serial streams were executing too rapidly and to capture this on oscilloscopes is nearly impossible and verify the data stream flow.

Resolution: Coded an internal Signal mapped to an external test point () that drives High the exact nanosecond serialization starts, and Low the moment it completes. This provided an exact idea when the stream starts and end.

CHAPTER 6: CONCLUSION AND FUTURE SCOPES

Conclusion:

Formal verification was performed to confirm that the internal design logic is functioning correctly without any problems. It is following the timing constraints mentioned in MRAM HXNV06400 datasheet. After successfully completing these simulations, the architecture of the complete system has been written and prototyped on an actual FPGA hardware board.

During hardware prototyping, we achieved stable data streaming at the 3 MHz design clock and 1.5 MHz configuration frequency. The oscilloscope tests proved that the physical clock signals work properly.

Future Scope:

In future we can update the system such that it can work with boot files.

We can also add a feature to the system that lets us read back data from virtex-5 and can help us find mistakes when we are programming the system during the boot process.

This will make it easier to figure out what is going wrong with the boot process, for the system.

REFERENCES

[1] AMD Xilinx, "Virtex-5 FPGA Configuration User Guide," User Guide UG191, v3.1.

[Context: Governs the hardware execution parameters for Slave Serial mode configuration, v5_cclk timing metrics, and the physical behavior of the v5_prog_b and v5_done pins].

[2] Xilinx, "ISE Design Suite Command Line Tools User Guide," Reference Guide UG628.

[Context: Referenced for native synthesis parameters, implementation workflows, and user constraint file (.ucf) pin mapping protocols for legacy Virtex-5 architectures].

[3] Honeywell Aerospace, "HXNV06400 64-Megabit (4M x 16 or 8M x 8) High-Reliability Magnetic Random Access Memory Datasheet," Revision H.

[Context: Utilized for non-volatile memory mapping, timing verification of parallel address latching, chip-select decoding matrices, and data bus orientation].

[4] IEEE Computer Society, "IEEE Standard for Verilog Hardware Description Language," IEEE Std 1364-2005, Standard IEEE Language Reference Manual (LRM).

[5] Palnitkar, S., *Verilog HDL: A Guide to Digital Design and Synthesis*, 2nd ed. Upper Saddle River, NJ: Prentice Hall PTR.

ANNEXURE

CODE:

Top Wrapper Bootloader (Top.v file) :

```
`timescale 1ns / 1ps

module v5_bootloader_top (
    input  wire clk,
    input  wire rst_n,
    input  wire start_boot,

    output wire [20:0] mram_a,
    output wire [3:0]  mram_ce_b,
    output wire mram_we,
    input  wire [7:0]  mram_dq,

    output reg  v5_prog_b,
    input  wire v5_done,
    output wire v5_cclk,
    output wire v5_din,

    output wire dbg_clk_3mhz,
    output wire dbg_dbg_rst_n,
    output wire dbg_byte_pulse
);

    wire rst;
    wire clk_3MHz;
    assign rst = ~rst_n;

    reg [1:0] clk_div;

    always @(posedge clk or posedge rst) begin
```

```
    if (rst) clk_div <= 2'd0;
    else     clk_div <= clk_div + 1'b1;
end
assign dbg_clk_3mhz = clk_div[1];
assign clk_3MHz = clk_div[1];

wire [7:0] pipe_data;
wire       pipe_ready;
wire       pipe_stall;
reg        dbg_rst_n;
wire       internal_mram_eof;
mram_interface mram_inst (
    .clk(clk_3MHz),
    .rst(rst),
    .start_config(dbg_rst_n),
    .mram_a(mram_a),
    .mram_ce_b(mram_ce_b),
    .mram_we(mram_we),
    .mram_dq(mram_dq),
    .tx_buffer_full(pipe_stall),
    .byte_out(pipe_data),
    .byte_ready(pipe_ready),
    .mram_eof(internal_mram_eof)
);
piso_serializer piso_inst (
    .clk(clk_3MHz),
    .rst(rst),
    .byte_in(pipe_data),
    .byte_ready(pipe_ready),
    .tx_buffer_full(pipe_stall),
    .v5_cclk(v5_cclk),
    .v5_din(v5_din),
```

```
        .byte_done_pulse(dbg_byte_pulse) // WIRING PISO TO EXTERNAL PIN
    );

//for MRAM Clock Generation after FPGA configured
 STARTUP_VIRTEX5 STARTUP_VIRTEX5_inst(
          .CFGCLK(), //-- Config logic clock 1-bit output
          .CFGMCLK(),// -- Config internal osc clock 1-bit output
          .DINSPI(), //-- DIN SPI PROM access 1-bit output
          .EOS(), //-- End of Startup 1-bit output
          .TCKSPI(), //-- TCK SPI PROM access 1-bit output
          .CLK(1'b0), //-- Clock input for start-up sequence
          .GSR(1'b0), //-- Global Set/Reset input (GSR cannot be used for the port name)
          .GTS(1'b0), //-- Global 3-state input (GTS cannot be used for the port name)
          .USRCCLKO(clk_3MHz), //-- User CCLK 1-bit input
          .USRCCLKTS(1'b0), //-- User CCLK 3-state, 1-bit input
          .USRDONEO(1'b1), //-- User Done 1-bit input
          .USRDONETS(1'b0) //-- User Done 3-state, 1-bit input
        );
    localparam IDLE         = 2'd0,
               WAIT_BUTTON  = 2'd1,
               STREAM_DATA  = 2'd2,
               WAKEUP_DELAY = 2'd3;

    reg [1:0] state;

    // THE ENVELOPE SIGNAL (SCOPE CH1)
    assign dbg_dbg_rst_n = dbg_rst_n;

    always @(posedge clk_3MHz or posedge rst) begin
        if (rst) begin
            state       <= IDLE;
            v5_prog_b   <= 1'b1;
```

```
            dbg_rst_n <= 1'b0;
        end else begin
            case (state)
                IDLE: begin
                    v5_prog_b <= 1'b0;
                    state     <= WAIT_BUTTON;
                end

                WAIT_BUTTON: begin
                    if (start_boot == 1'b1) begin
                        v5_prog_b   <= 1'b1;
                        dbg_rst_n <= 1'b1;
                        state       <= STREAM_DATA;
                    end
                end

                STREAM_DATA: begin
                    if (v5_done == 1'b1 || internal_mram_eof == 1'b1) begin
                        dbg_rst_n <= 1'b0;
                        state       <= WAKEUP_DELAY;
                    end
                end

                WAKEUP_DELAY: begin
                    // Hold state indefinitely
                end

                default: state <= IDLE;
            endcase
        end
    end
endmodule
```

MRAM Interface (mram_interface.v file) :

```
`timescale 1ns / 1ps
module mram_interface (
    input  wire clk,
    input  wire rst,
    input  wire start_config,

    output reg  [20:0] mram_a,
    output reg  [3:0]  mram_ce_b,
    output wire mram_we,
    input  wire [7:0]  mram_dq,

    input  wire tx_buffer_full,
    output reg  [7:0]  byte_out,
    output reg  byte_ready,
    output reg  mram_eof
);

    assign mram_we = 1'b0;

    localparam IDLE       = 2'd0,
               S_PRIME    = 2'd1,
               S_STREAM   = 2'd2,
               S_WAIT_ACK = 2'd3;

    reg [1:0]  state;
    reg [22:0] sys_addr;
```

```
reg active_mission;

wire [22:0] next_addr = sys_addr + 1'b1;

always @(posedge clk or posedge rst) begin
   if (rst) begin
      state          <= IDLE;
      sys_addr       <= 23'd0;
      mram_a         <= 21'd0;
      mram_ce_b      <= 4'b1111;
      byte_ready     <= 1'b0;
      byte_out       <= 8'h00;
      active_mission <= 1'b0;
      mram_eof       <= 1'b0;
   end else begin
      byte_ready <= 1'b0;
      mram_eof   <= 1'b0;

      if (start_config && !active_mission) begin
         active_mission <= 1'b1;
         sys_addr       <= 23'd0;
         state          <= S_PRIME;
      end
      else if (active_mission) begin
         case (state)

            S_PRIME: begin
               mram_a    <= sys_addr[20:0];
               mram_ce_b <= 4'b1110;
               state     <= S_STREAM;
            end
```

```
S_STREAM: begin
   if (!tx_buffer_full) begin

      byte_out   <= mram_dq;
      byte_ready <= 1'b1;

      if (sys_addr == 23'h7FFFFF) begin
         active_mission <= 1'b0;
         mram_ce_b      <= 4'b1111;
         mram_eof       <= 1'b1;
         state          <= IDLE;
      end else begin
         mram_a <= next_addr[20:0];

         case (next_addr[22:21])
            2'b00: mram_ce_b <= 4'b1110;
            2'b01: mram_ce_b <= 4'b1101;
            2'b10: mram_ce_b <= 4'b1011;
            2'b11: mram_ce_b <= 4'b0111;
         endcase

         sys_addr <= next_addr;
         state    <= S_WAIT_ACK;
      end
   end
end

S_WAIT_ACK: begin
   if (tx_buffer_full == 1'b1) begin
      state <= S_STREAM;
   end
end
```

```
            endcase
          end
        end
    end
endmodule
```

PISO Serializer (piso.v file) :

```
`timescale 1ns / 1ps
module piso_serializer (
    input  wire clk,          // 100 MHz Master
    input  wire rst,
    input  wire [7:0] byte_in,
    input  wire byte_ready,
    output wire tx_buffer_full,
    output reg  v5_cclk,      // True 50 MHz Gated Clock
    output reg  v5_din,
    output reg byte_done_pulse
);

    reg [7:0] shadow_buffer;
    reg       shadow_valid;
    reg [7:0] shift_reg;
    reg [2:0] bit_cnt;
    reg       is_shifting;
    reg       phase;          // 0 = Drive Data (Low CCLK), 1 = Sample Data (High CCLK)

    assign tx_buffer_full = shadow_valid;

    always @(posedge clk or posedge rst) begin
        if (rst) begin
```

```
        shadow_buffer     <= 8'h00;
        shadow_valid      <= 1'b0;
        shift_reg         <= 8'h00;
        bit_cnt           <= 3'd0;
        is_shifting       <= 1'b0;
        phase             <= 1'b0;
        v5_cclk           <= 1'b0;
        v5_din            <= 1'b0;
                                byte_done_pulse   <= 1'b0;
    end else begin
        byte_done_pulse   <= 1'b0;
        // 1. THE INTAKE (Catches data instantly from MRAM)
        if (byte_ready && !shadow_valid) begin
            shadow_buffer <= byte_in;
            shadow_valid  <= 1'b1;
        end

        // 2. THE EXHAUST (True 50 MHz Phase-Locked Engine)
        if (is_shifting) begin

            if (phase == 1'b0) begin
                // PHASE 0: The Falling Edge.
                // Safe to change data while Virtex-5 clock is low.
                v5_cclk   <= 1'b0;
                v5_din    <= shift_reg[7];
                shift_reg <= {shift_reg[6:0], 1'b0};
                phase     <= 1'b1;
            end
            else begin
                // PHASE 1: The Rising Edge.
                // Virtex-5 physically samples v5_din right now.
                v5_cclk <= 1'b1;
```

```
            phase   <= 1'b0;

            if (bit_cnt == 3'd7) begin
                                                    byte_done_pulse <= 1'b1;
               // Byte is complete. Check if the shadow buffer has the NEXT byte.
               if (shadow_valid) begin
                  // SEAMLESS HANDOFF: Load next byte instantly to maintain 100%
bandwidth.
                  shift_reg    <= shadow_buffer;
                  shadow_valid <= 1'b0;
                  bit_cnt      <= 3'd0;
                  // is_shifting remains 1, phase naturally rolls over to 0 next tick.
               end else begin
                  is_shifting <= 1'b0; // Pipeline starved. Shut down.
               end
            end else begin
               bit_cnt <= bit_cnt + 1'b1;
            end
         end

      end
      else if (shadow_valid) begin
         // WAKING UP FROM IDLE
         shift_reg    <= shadow_buffer;
         shadow_valid <= 1'b0;
         is_shifting  <= 1'b1;
         bit_cnt      <= 3'd0;
         phase        <= 1'b0;
         v5_cclk      <= 1'b0; // Ensure clock is clamped before shifting begins
      end
      else begin
         // THE DEAD GAP (Gated Clock)
```

```
            // If pipeline is starved, clock stays parked Low. Zero false edges.
            v5_cclk <= 1'b0;
        end

    end
  end
endmodule
```

Top Wrapper Bootloader Testbench (tb_Top.v file) :

```
`timescale 1ns / 1ps
module tb_v5_bootloader;
  // Inputs
  reg clk;
  reg rst_n;
  reg start_boot;
  reg v5_done;

  // Outputs
  wire [20:0] mram_a;
  wire [3:0]  mram_ce_b;
  wire mram_we;
  wire v5_prog_b;
  wire v5_cclk;
  wire v5_din;
  wire dbg_clk_3mhz;
  wire dbg_dbg_rst_n;

  // Bidirectional
  wire [7:0] mram_dq;
```

```
    // Instantiate the Unit Under Test (UUT)
    v5_bootloader_top uut (
        .clk(clk),
        .rst_n(rst_n),
        .start_boot(start_boot),
        .mram_a(mram_a),
        .mram_ce_b(mram_ce_b),
        .mram_we(mram_we),
        .mram_dq(mram_dq),
        .v5_prog_b(v5_prog_b),
        .v5_done(v5_done),
        .v5_cclk(v5_cclk),
        .v5_din(v5_din),
        .dbg_clk_3mhz(dbg_clk_3mhz),
        .dbg_dbg_rst_n(dbg_dbg_rst_n)
    );
    // 12 MHz MASTER CLOCK (Period ~83.33 ns)
    always #41.67 clk = ~clk;
    // MOCK MRAM
    assign mram_dq = (mram_ce_b == 4'b1110 && mram_a == 21'h000000) ? 8'hA1 :
                     (mram_ce_b == 4'b1110 && mram_a == 21'h000001) ? 8'hB2 :
                     (mram_ce_b == 4'b1110 && mram_a == 21'h000002) ? 8'hC3 :
                     (mram_ce_b == 4'b0111 && mram_a == 21'h7FFFFF) ? 8'h99 : 8'hZZ;
    initial begin

$display("\n=====================================================");
        $display("   TESTBENCH: ENVELOPE SIGNAL VERIFICATION ");

$display("=====================================================\n");


        // Initial State
        clk = 0;
```

```
        rst_n = 0;
        start_boot = 0;
        v5_done = 0;

        #200;
        rst_n = 1;

        // 1. Check initial state
        wait(v5_prog_b == 1'b0);
        $display("[TIME: %0t] POWER UP: v5_prog_b pulled low.", $time);
        $display("[TIME: %0t] ENVELOPE CHECK: dbg_rst_n is %b (Should be 0)", $time, dbg_rst_n);

        #2000;

        // 2. Press the button
        $display("\n[TIME: %0t] Lab engineer pressing start_boot...", $time);
        start_boot = 1;
        #100;
        start_boot = 0;


        // 3. Verify envelope opens
        wait(dbg_rst_n == 1'b1);
        $display("[TIME: %0t] ENVELOPE OPENED: dbg_rst_n spiked HIGH! Pipeline is active.", $time);

        // Let the pipeline run and shift out data
        #50000;

        // 4. Virtex-5 finishes and raises v5_done
        $display("\n[TIME: %0t] Target Virtex-5 asserting v5_done...", $time);
        v5_done = 1;
```

```
        // 5. Verify envelope closes
        wait(dbg_rst_n == 1'b0);
        $display("[TIME: %0t] ENVELOPE CLOSED: dbg_rst_n dropped LOW! Mission complete.", $time);

        #500;

$display("\n============================================================");
        $display("   TESTBENCH COMPLETE ");

$display("============================================================\n");
        $finish;
    end
endmodule
```

UCF (User Constraint File) Code :

```
# VIRTEX-5 BOOTLOADER CONSTRAINTS FILE
# MAPS v5_bootloader_top.v PORTS TO PHYSICAL OBSP BOARD PINS
# ------------------------------------------------------------------------
# SYSTEM CLOCK & RESET
# ------------------------------------------------------------------------
# Assuming AL15 provides the single-ended system clock.
# WARNING: Verify physical frequency. If not 100MHz, a PLL/DCM is required.
NET "clk" LOC = "J16" | IOSTANDARD = "LVCMOS33";
NET "dbg_clk_3mhz" LOC = "P7" | IOSTANDARD = "LVCMOS33";


# Map Reset to the Power-On-Reset Test Point
NET "rst_n"     LOC = "AU22"  | IOSTANDARD = "LVCMOS33";
NET "dbg_rst_n" LOC = "E8"    | IOSTANDARD = "LVCMOS33";

# THE LAB TRIGGER (Manual Ignition)
# PULLDOWN is mandatory so floating static doesn't fire the bootloader
```

```
NET "start_boot" LOC = "K5" | IOSTANDARD = "LVCMOS33" | PULLDOWN; ###K1-76


# ------------------------------------------------------------------
# MRAM PHYSICAL INTERFACE
# ------------------------------------------------------------------
# Address Bus (23 bits)
NET "mram_a[0]"  LOC = "P13"  | IOSTANDARD = "LVCMOS33";
NET "mram_a[1]"  LOC = "N13"  | IOSTANDARD = "LVCMOS33";
NET "mram_a[2]"  LOC = "M29"  | IOSTANDARD = "LVCMOS33";
NET "mram_a[3]"  LOC = "N30"  | IOSTANDARD = "LVCMOS33";
NET "mram_a[4]"  LOC = "M13"  | IOSTANDARD = "LVCMOS33";
NET "mram_a[5]"  LOC = "M14"  | IOSTANDARD = "LVCMOS33";
NET "mram_a[6]"  LOC = "N29"  | IOSTANDARD = "LVCMOS33";
NET "mram_a[7]"  LOC = "N28"  | IOSTANDARD = "LVCMOS33";
NET "mram_a[8]"  LOC = "N14"  | IOSTANDARD = "LVCMOS33";
NET "mram_a[9]"  LOC = "N15"  | IOSTANDARD = "LVCMOS33";
NET "mram_a[10]" LOC = "P28"  | IOSTANDARD = "LVCMOS33";
NET "mram_a[11]" LOC = "P27"  | IOSTANDARD = "LVCMOS33";
NET "mram_a[12]" LOC = "N16"  | IOSTANDARD = "LVCMOS33";
NET "mram_a[13]" LOC = "M16"  | IOSTANDARD = "LVCMOS33";
NET "mram_a[14]" LOC = "N26"  | IOSTANDARD = "LVCMOS33";
NET "mram_a[15]" LOC = "P26"  | IOSTANDARD = "LVCMOS33";
NET "mram_a[16]" LOC = "P17"  | IOSTANDARD = "LVCMOS33";
NET "mram_a[17]" LOC = "P18"  | IOSTANDARD = "LVCMOS33";
NET "mram_a[18]" LOC = "P25"  | IOSTANDARD = "LVCMOS33";
NET "mram_a[19]" LOC = "N25"  | IOSTANDARD = "LVCMOS33";
NET "mram_a[20]" LOC = "AM29" | IOSTANDARD = "LVCMOS33";
# Data Bus (8 bits)
NET "mram_dq[0]" LOC = "AJ26" | IOSTANDARD = "LVCMOS33";
NET "mram_dq[1]" LOC = "AK27" | IOSTANDARD = "LVCMOS33";
NET "mram_dq[2]" LOC = "AM14" | IOSTANDARD = "LVCMOS33";
NET "mram_dq[3]" LOC = "AN14" | IOSTANDARD = "LVCMOS33";
```

```
NET "mram_dq[4]" LOC = "AK29" | IOSTANDARD = "LVCMOS33";
NET "mram_dq[5]" LOC = "AK28" | IOSTANDARD = "LVCMOS33";
NET "mram_dq[6]" LOC = "AP13" | IOSTANDARD = "LVCMOS33";
NET "mram_dq[7]" LOC = "AN13" | IOSTANDARD = "LVCMOS33";

# MRAM Control Pins
NET "mram_ce_b[0]"  LOC = "AL30" | IOSTANDARD = "LVCMOS33";
NET "mram_ce_b[1]"  LOC = "AL14" | IOSTANDARD = "LVCMOS33";
NET "mram_ce_b[2]"  LOC = "AM28" | IOSTANDARD = "LVCMOS33";
NET "mram_ce_b[3]"  LOC = "AM13" | IOSTANDARD = "LVCMOS33";
NET "mram_we"       LOC = "AK14" | IOSTANDARD = "LVCMOS33";




# ------------------------------------------------------------------
# VIRTEX-5 TARGET CONFIGURATION (SLAVE SERIAL)
# ------------------------------------------------------------------
# The Gated Clock and Data Stream
NET "v5_cclk"   LOC = "V35"  | IOSTANDARD = "LVCMOS33" | SLEW = FAST |
DRIVE = 12;
NET "v5_din"    LOC = "P35"  | IOSTANDARD = "LVCMOS33" | SLEW = FAST | DRIVE
= 12;



# Handshake Pins
NET "v5_prog_b" LOC = "L35"  | IOSTANDARD = "LVCMOS33";
# NET "v5_init_b" LOC = "H35"  | IOSTANDARD = "LVCMOS33";
NET "v5_done"   LOC = "E35"  | IOSTANDARD = "LVCMOS33";

# TIMING CONSTRAINT
# Force the synthesizer to respect the 100MHz master clock period
NET "clk" TNM_NET = "sys_clk";
TIMESPEC TS_sys_clk = PERIOD "sys_clk" 83.33 ns HIGH 50%;
```